



AMRITA

VISHWA VIDYAPEETHAM

Course Code: 23ECE333

Course Name: Functional Verification

Component: Simulation-based Assignment

Date of Evaluation:

Academic Year: 2025-2026 (Even Semester)

Title: Design and Verification of FSM-Based Weighted Round-Robin Arbiter Using SystemVerilog

Batch Number:12

Name of the Students With Registration no.:

- 1) M.Madhumitha - BL.EN.U4ECE23226
- 2) V Sivani - BL.EN.U4ECE23253
- 3) P Ysaswini - BL.EN.U4ECE23254

Branch: ECE

Semester: VI Semester

Batch: 2023-2027

Faculty In-charge: Swaminadhan Rajula

Aim:

The aim of this project is to design a Weighted Round-Robin (WRR) Arbiter that allows multiple bus masters to share system resources efficiently and fairly. The arbiter provides access to requestors in a round-robin manner while also supporting adjustable weights so that higher-priority requestors can receive more access time compared to others. This method improves the overall system performance and throughput while preventing starvation of any requestor. The design distributes resource access based on the assigned weights and priorities of the requestors. The functionality and performance of the arbiter are verified using SystemVerilog with randomized test inputs and coverage analysis.

Base paper Details:

Title: Design and Verification of FSM-Based Weighted Round-Robin Arbiter Using SystemVerilog

Authors: Mr. Mark A. Indovina & Dr. Sohail A. Dianat

Tool Used: Xilinx Vivado

Theory:

1) Introduction:

A Weighted Round-Robin (WRR) Arbiter is used in digital systems to manage access to shared resources among multiple bus masters. It provides fair resource allocation by granting access in a round-robin sequence while also considering assigned weights for each requestor. The arbiter prevents starvation and improves overall system throughput and efficiency. An FSM-based control mechanism is used to manage grant generation and weight counting operations. The design and verification of the arbiter are implemented using Verilog/system Verilog for reliable hardware operation.

2) Working principle:

Multiple requestors send access requests to the arbiter simultaneously. The FSM checks the current grant and determines the next requestor using round-robin logic. The Weight Decoder fetches the assigned weight of the selected requestor. The selected requestor receives the grant for the number of clock cycles equal to its weight. After completion, the arbiter moves to the next requestor and repeats the process fairly.

3) Advantages:

Ensures fair resource allocation among all requestors.
Prevents starvation by following round-robin scheduling.
Supports configurable weights for priority-based bandwidth allocation.
Improves overall throughput and bus utilization in SoC systems.
Suitable for high-speed communication, packet switching, and memory arbitration applications.

4) Disadvantages:

Increased hardware complexity due to FSM and weight management logic.
Area and power overhead increase with higher number of requestors.
Verification becomes more complex for large-scale arbitration systems.
Additional latency may occur because of weight-based scheduling operations.
Dynamic priority handling is limited compared to adaptive arbitration techniques

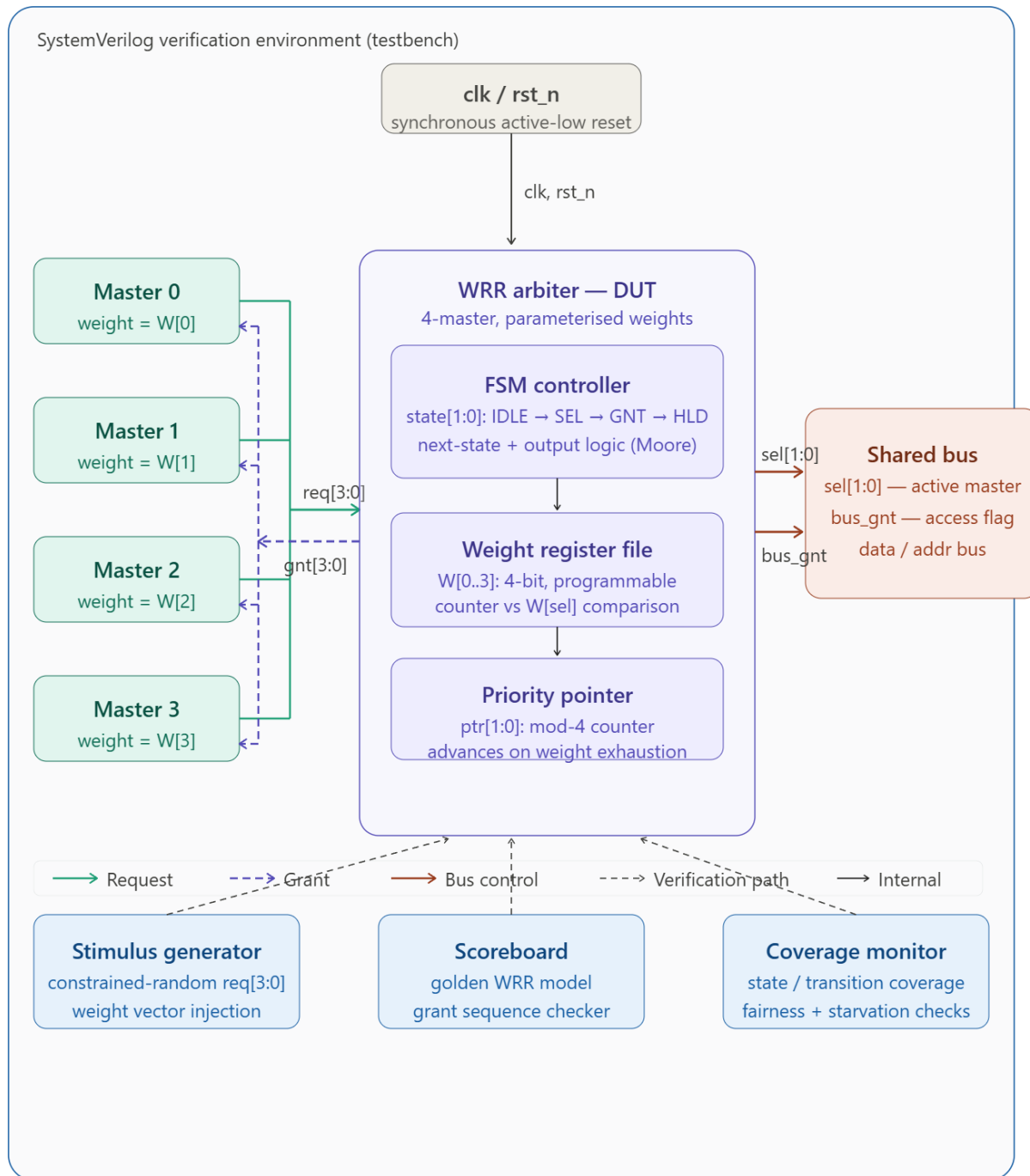
5) Block Diagram:

The system consists of Request Inputs, Weight Decoder, Next Grant Precalculator (NGPRC), FSM-Based Grant State Machine, and Grant Outputs.

The request inputs are provided by multiple bus masters requesting access to a shared resource. The Weight Decoder determines how long each requestor can hold the grant based on assigned weights.

The NGPRC calculates the next eligible requestor in round-robin order to maintain fairness.

Finally, the FSM-based Grant State Machine controls grant allocation and generates the final grant outputs.



SystemVerilog Code for Design Block:

```
`timescale 1ns/1ps
module weighted_rr_arbiter #(
    parameter N = 4,
    parameter W = 4
)(
    input logic clk,
    input logic reset,
    input logic [N-1:0] req,
    input logic [N*W-1:0] weight_bus,
    output logic [N-1:0] gnt
);
    localparam IDLE = 2'b00,
               GRANT = 2'b01,
               COUNT = 2'b10,
               UPDATE = 2'b11;
    logic [1:0] state, next_state;
    logic [$clog2(N)-1:0] pointer;
    logic [$clog2(N)-1:0] sel;
    logic [W-1:0] weight;
    logic [W-1:0] counter;
    integer i;
    integer idx;

    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            state <= IDLE;
        else
            state <= next_state;
    end

    always_comb begin
        case (state)
            IDLE: begin
                if (req != 0)
                    next_state = GRANT;
                else
                    next_state = IDLE;
            end

            GRANT: begin
                next_state = COUNT;
            end

            COUNT: begin
                if (counter < weight)
                    next_state = COUNT;
                else
                    next_state = UPDATE;
            end

            UPDATE: begin
                next_state = IDLE;
            end
        endcase
    end
end
```

```

        UPDATE: begin
            if (req != 0)
                next_state = GRANT;
            else
                next_state = IDLE;
            end
        end

        default: next_state = IDLE;
    endcase
end

always_comb begin
    sel = pointer;
    for (i = 0; i < N; i = i + 1) begin
        idx = (pointer + i) % N;

        if (req[idx]) begin
            sel = idx[$clog2(N)-1:0];
            break;
        end
    end
end

always_comb begin
    weight = weight_bus[sel*W +: W];
end

always_ff @(posedge clk or posedge reset) begin
    if (reset) begin
        gnt    <= 0;
        pointer <= 0;
        counter <= 0;
    end
    else begin
        case (state)

            IDLE: begin
                gnt    <= 0;
                counter <= 0;
            end

            GRANT: begin
                gnt    <= 0;
                gnt[sel] <= 1'b1;
                counter <= 1;
            end

            COUNT: begin
                gnt    <= 0;
                gnt[sel] <= 1'b1;
                counter <= counter + 1;
            end

            UPDATE: begin
                gnt    <= 0;
                counter <= 0;

```

```

if (sel == N-1)
    pointer <= 0;
else
    pointer <= sel + 1'b1;
end

default: begin
    gnt <= 0;
    counter <= 0;
end
endcase
end
end
endmodule

```

SystemVerilog Code for Layered Testbench:

```

`timescale 1ns/1ps
interface rr_if #(parameter N = 4, parameter W = 4) (input logic clk);

    logic reset;
    logic [N-1:0] req;
    logic [N*W-1:0] weight_bus;
    logic [N-1:0] gnt;
endinterface

class transaction;
    rand bit [3:0] req;
    rand bit [15:0] weight_bus;
    function void display(string name);
        $display("[%s] req=%b weight_bus=%h", name, req, weight_bus);
    endfunction
endclass

class generator;
    mailbox #(transaction) gen2drv;
    function new(mailbox #(transaction) gen2drv);
        this.gen2drv = gen2drv;
    endfunction

    task run();
        transaction tr;
        tr = new();
        tr.req = 4'b1111;
        tr.weight_bus = 16'h2132;
        gen2drv.put(tr);
        tr = new();
        tr.req = 4'b1010;
        tr.weight_bus = 16'h2132;
        gen2drv.put(tr);
        tr = new();
        tr.req = 4'b0000;
        tr.weight_bus = 16'h2132;
        gen2drv.put(tr);
    endtask
endclass

```

```

class driver;
  virtual rr_if vif;
  mailbox #(transaction) gen2drv;
  function new(virtual rr_if vif, mailbox #(transaction) gen2drv);
    this.vif = vif;
    this.gen2drv = gen2drv;
  endfunction

  task reset_dut();
    vif.reset = 1;
    vif.req = 4'b0000;
    vif.weight_bus = 16'h0000;
    repeat (3) @(posedge vif.clk);
    vif.reset = 0;
    $display("[DRIVER] Reset completed");
  endtask

  task run();
    transaction tr;
    forever begin
      gen2drv.get(tr);
      @(negedge vif.clk);
      vif.req = tr.req;
      vif.weight_bus = tr.weight_bus;
      tr.display("DRIVER");
      repeat (40) @(posedge vif.clk);
    end
  endtask
endclass

class monitor;
  virtual rr_if vif;
  mailbox #(transaction) mon2scb;
  function new(virtual rr_if vif, mailbox #(transaction) mon2scb);
    this.vif = vif;
    this.mon2scb = mon2scb;
  endfunction

  task run();
    transaction tr;
    forever begin
      @(posedge vif.clk);
      tr = new();
      tr.req = vif.req;
      tr.weight_bus = vif.weight_bus;
      mon2scb.put(tr);
      $display("[MONITOR] time=%0t req=%b weight=%h gnt=%b",
        $time, vif.req, vif.weight_bus, vif.gnt);
    end
  endtask
endclass

```

```

class scoreboard;
    virtual rr_if vif;
    mailbox #(transaction) mon2scb;
    function new(virtual rr_if vif, mailbox #(transaction) mon2scb);
        this.vif = vif;
        this.mon2scb = mon2scb;
    endfunction

    task run();
        transaction tr;
        forever begin
            mon2scb.get(tr);
            if (!(vif.gnt == 4'b0000 ||
                vif.gnt == 4'b0001 ||
                vif.gnt == 4'b0010 ||
                vif.gnt == 4'b0100 ||
                vif.gnt == 4'b1000)) begin
                $error("[SCOREBOARD] ERROR: Multiple grants active! gnt=%b", vif.gnt);
            end
            if ((vif.gnt & vif.req) != vif.gnt) begin
                $error("[SCOREBOARD] ERROR: Grant given without request! req=%b gnt=%b",
                    vif.req, vif.gnt);
            end
            if (vif.req == 4'b0000 && vif.gnt != 4'b0000) begin
                $error("[SCOREBOARD] ERROR: Grant active when no request!");
            end
        end
    endtask
endclass

class environment;
    generator gen;
    driver drv;
    monitor mon;
    scoreboard scb;
    mailbox #(transaction) gen2drv;
    mailbox #(transaction) mon2scb;
    virtual rr_if vif;
    function new(virtual rr_if vif);
        this.vif = vif;
        gen2drv = new();
        mon2scb = new();
        gen = new(gen2drv);
        drv = new(vif, gen2drv);
        mon = new(vif, mon2scb);
        scb = new(vif, mon2scb);
    endfunction

    task run();
        drv.reset_dut();
        fork
            gen.run();
            drv.run();
            mon.run();
            scb.run();
        join_any
    endtask
endclass

```



```

module tb_weighted_rr;
    parameter N = 4;
    parameter W = 4;
    logic clk = 0;
    always #5 clk = ~clk;
    rr_if #(N, W) vif(clk);

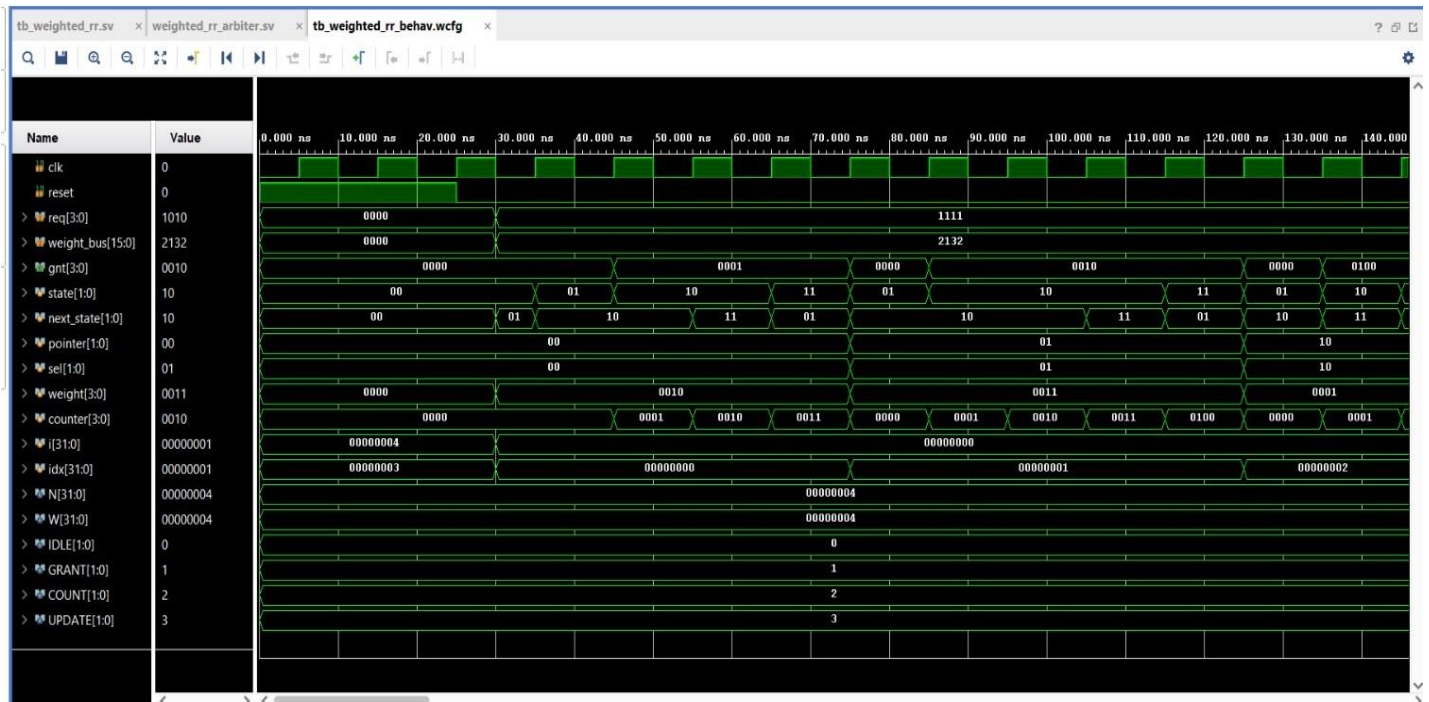
    weighted_rr_arbiter #(N, W) dut (
        .clk(clk),
        .reset(vif.reset),
        .req(vif.req),
        .weight_bus(vif.weight_bus),
        .gnt(vif.gnt)
    );

    environment env;
    initial begin
        env = new(vif);
        env.run();

        #1000;
        $finish;
    end
endmodule.

```

Simulation Results (Proper screenshots):



Conclusion:

The FSM-based Weighted Round-Robin Arbiter provides an efficient and fair mechanism for allocating shared resources among multiple requestors in SoC systems. The arbiter successfully prevents starvation while supporting configurable weights for priority-based access control. The integration of Weight Decoder, Next Grant Precalculator, and FSM-based Grant State Machine improves overall system throughput and resource utilization. Functional verification using SystemVerilog confirms the correct operation of the arbitration and grant allocation process. Thus, the proposed design offers a reliable and scalable solution for modern bus arbitration and communication systems.

References:

- Aung Toe, "Design and Verification of a Round-Robin Arbiter," Rochester Institute of Technology, 2018.
- C. Spear, SystemVerilog for Verification: A Guide to Learning the Testbench Language Features, Springer, 2006.
- E. S. Shin, V. J. Mooney, and G. F. Riley, "Round-robin Arbiter Design and Generation," IEEE, 2002

